

```
c = red;           // OK
c = 1;             // Also OK, means the same
```

With explicit integral initialisers, you can set one or more enumerators to specific values. The initialiser can be any expression yielding a positive or negative integer value (after possible integer promotions). Any subsequent names without initialisers will then increase by one. These values are usually unique, but duplicates are legal. The order of constants can be explicitly re-arranged. For example:

```
enumcolors { black,      // value 0
            red,         // value 1
            green,       // value 2
            blue=6,      // value 6
            violet,      // value 7
            white=4 };   // value 4
```

Initialiser expression can include previously declared enumerators. For example, in the following declaration:

```
enummemory_sizes { bit = 1, nibble = 4 * bit, byte = 2
* nibble,
                    kilobyte = 1024 * byte };
```

nibble would acquire the value 4, byte the value 8, and kilobyte the value 8192.

Anonymous Enum Type

In our previous declaration, the identifier `colors` is the optional enumeration tag that can be used in subsequent declarations of enumeration variables of type `enumcolors`:

```
enumcolorsbg, border; /* declare variables bg and
border */
```

As with struct and union declarations, you can omit the tag if no further variables of this enum type are required:

```
/* Anonymous enum type: */
enum { black, red, green, blue, violet, white } color;
```

Enumeration Scope

Enumeration tags share the same name space as structure and union tags. Enumerators share the same name space as ordinary variable identifiers:

```
int blue = 73;
{ // open a block
```